# PS2 Static Recompilation — Architecture Overview

> A reference for how a PS2 ELF becomes a native host binary in this project (PS2Recomp / `ps2xRuntime`), what each class represents, and how the generated function code is linked into a runnable build.
>
> Reference repo: **Tales of Legendia (`SLUS_212.01`)**.

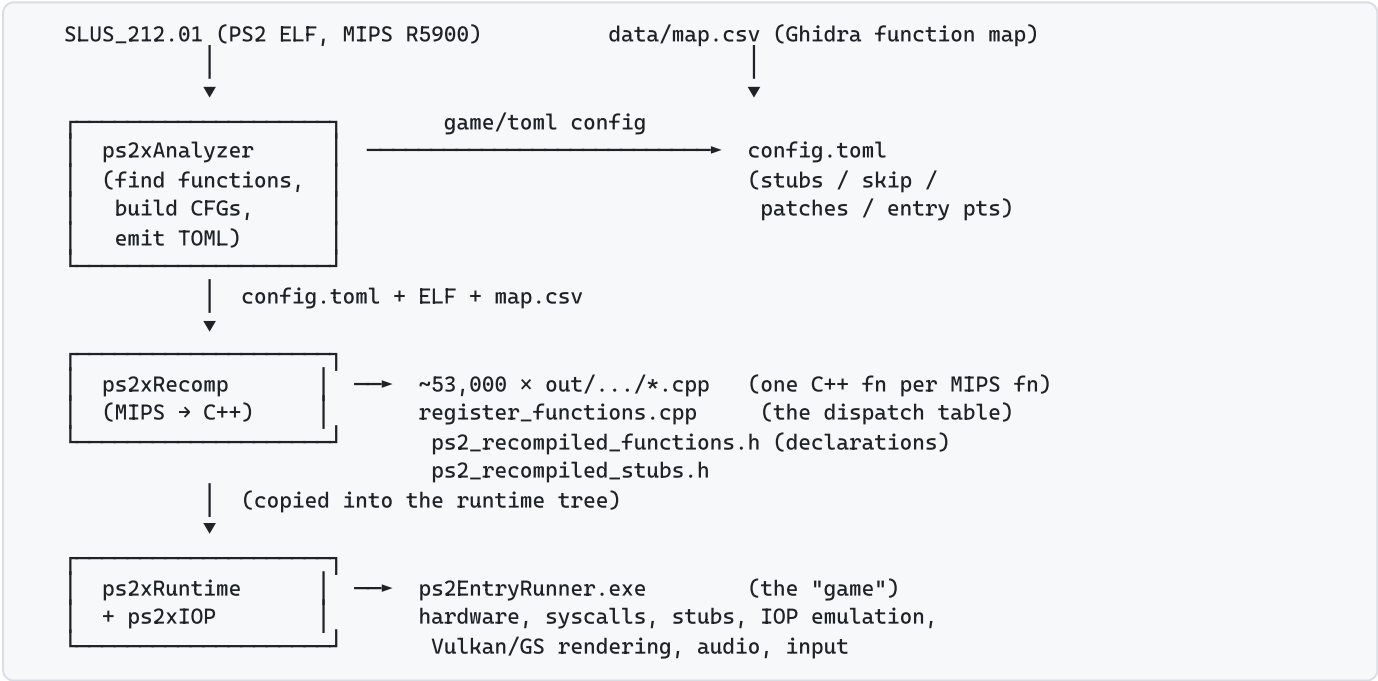
---

## 1. What "static recompilation" means here

Emulation reads a MIPS instruction, decodes it, and executes it in an interpreter loop at runtime. **Static recompilation translates the MIPS ahead of time into C++**, compiles that C++ with a normal host compiler, and links it into the app. There is no MIPS interpreter on the hot path — the game's functions *are* native host functions.

Trade-off: an emulator runs any disc; a recompiler produces a binary **for one specific ELF**, and anything the translator or the hardware model does not cover must be filled in by hand.

The repo is three programs plus a runtime library:




`ps2xIOP` is a fourth piece: the PS2's I/O processor runs as a **separate emulated subsystem** (it interprets the original `.IRX` modules), because IOP code is not statically recompiled the way EE code is.

---

## 2. Stage 1 — The Analyzer (`ps2xAnalyzer/`)

Goal: enumerate every function boundary so the recompiler knows what to translate, and emit `config.toml`.

| Class | File | Represents |
|---|---|---|
| `ElfAnalyzer` | `src/elf_analyzer.cpp` | The orchestrator. Loads the ELF, decodes all functions, classifies them, runs analysis passes, writes the TOML (`analyze()`, `generateToml()`, `importGhidraMap()`). |
| `ElfAnalysisContext` | `src/elf_analysis_context.cpp` | Shared scratch state accumulated across passes (CFGs, call graph, data usage, patches). |
| `FunctionClassifier` | `src/function_classifier.cpp` | Decides what each function *is*: recompile / stub / skip / library-like. |
| `SceSymbolScanner` | `src/sce_symbol_scanner.cpp` | Recognises Sony SDK (`sce*`) names, backed by `sce_symbol_database_data.h`. |
| `AnalysisPasses` | `src/analysis_passes.cpp` | Static analyses: hardware-I/O signals, MMI complexity, self-modifying code, jump-table detection, recursion. |
| `TomlGenerator` | `src/toml_generator.cpp` | Serialises the result into `config.toml`. |
| `R5900Decoder` / `ElfParser` | shared with recompiler | Raw MIPS decode + ELF section/symbol/relocation parsing. |

Three discovery modes:

1. **DWARF symbols** — ideal, exact boundaries.
2. **Stripped** — heuristic `jal` scanner.
3. **Ghidra map** — for retail/protected games, export `map.csv` with `ExportPS2Functions.java` and feed it in.

This repo uses **mode 3**: `config.toml` sets `ghidra_output = data/map.csv`, `function_count = 11604`.

---

## 3. Stage 2 — The Recompiler (`ps2xRecomp/`)

Goal: translate each MIPS function into a C++ function with the signature

```
void f(uint8_t* rdram, R5900Context* ctx, PS2Runtime* runtime);
```

A MIPS function becomes a **straight-line C++ function** with `goto` labels for internal branches. From `entry_0x100008.cpp` (the ELF entry point):

```
void entry_0x100008(uint8_t* rdram, R5900Context* ctx, PS2Runtime *runtime) {
    switch (ctx->pc) { case 0x10012cu: goto label_10012c; ... default: break; }
    ctx->pc = 0x100008u;
    // 0x100008: 0x70000c28  padduw      $at, $zero, $zero
    SET_GPR_VEC(ctx, 1, ps2_paddu32(GPR_VEC(ctx, 0), GPR_VEC(ctx, 0)));
    ...
}
```

| Class | File | Represents |
|---|---|---|
| `PS2Recompiler` | `src/ps2_recompiler.cpp` | Top-level driver: reads config + ELF, drives generation, writes files. |
| `CodeGenerator` | `src/code_generator.cpp` | Turns a decoded function into C++; owns naming (`getFunctionName`) and per-function analysis. |

| Class | File | Represents |
|---|---|---|
| `FunctionEmitter` | `src/function_emitter.cpp` | Emits the body of one C++ function (labels, `switch(ctx->pc)` resume entries, per-instruction code). |
| `FunctionTableEmitter` | `src/function_table_emitter.cpp` | Emits **`register_functions.cpp`** — the flat dispatch table (see §5). |
| `ControlFlowAnalyzer` / `control_flow_emitter` | | Resolve branch targets, entry points, delay slots; emit branch / `dispatchGuestBranch` calls. |
| `JumpTableSwitchEmitter` | | Converts indirect jump tables into C++ `switch`. |
| `GifDmaKickAnalyzer` | | Detects DMA→GIF chain kicks so the runtime can render them. |
| `R5900Decoder` | `src/r5900_decoder.cpp` | Decodes raw instruction words. |
| **Translators** ( `instruction_` , `mmi_` , `vu_` , `fpu_` , `cop0_` , `regimm_` , `special_` ) | `src/lib/*translator.cpp` | Per-instruction-class MIPS→C++ lowering (MMI/SIMD, VU0 macro-mode, FPU, COP0, special encodings). |
| `ConfigManager` | | Parses `config.toml`. |
| `RecompilerReporter` | | Collects errors/warnings ( `errorAt(...)` ). |

### Config knobs that affect correctness

| Knob | Meaning |
|---|---|
| `stubs` | Functions to **not** translate; generate a thin wrapper calling a known runtime handler. Also accepts `handler@0xADDRESS` to bind a stripped address directly. |
| `skip` | Actively ignore; emit `ps2_stubs::TODO_NAMED(...)`. |
| `[patches].instructions` | Replace raw MIPS instructions before translation (e.g. nop out a spinlock). |
| `single_file_output` | One giant `.cpp` vs one file per function (this repo: `false` ). |
| `patch_syscalls` / `patch_cop0` / `patch_cache` | Apply configured patches to those instruction classes. |

If the translator hits an opcode it does not understand, it writes `// Unhandled opcode at 0x...` in the generated file. Those become game-override work.

Example stub bindings in this repo ( `data/config.toml` ):

```
CreateSema@0x001146E0
WaitSema@0x00114720
SignalSema@0x00114700
```

---

## 4. Stage 3 — The Runtime ( `ps2xRuntime/` )

This *is* the game executable. The recompiled C++ is compiled and linked into it. `src/main.cpp` is the actual `main()`:

```
PS2Runtime runtime;
runtime.initialize(windowTitle);   // window, GS backend, VU, IOP, audio, pad
runtime.loadELF(filePathStr);      // load guest image into RAM, apply overrides
runtime.run();                     // start scheduler + render loop
```

### 4a. Core

| Class | Header | Represents |
|---|---|---|
| `PS2Runtime` | `include/ps2_runtime.h` | **The hub.** Holds CPU context, memory, GS, VU, IOP, audio/pad/VFS; owns function-table dispatch ( `lookupFunction` , `replaceFunction` , `dispatchGuestBranch` ); implements `Load*/Store*` for guest memory; syscall/exception entry. |
| `R5900Context` | `ps2_runtime.h` | CPU register file: 32 × `__m128i` GPRs, HI/LO/HI1/LO1, PC, COP0 registers, VU0 macro-mode state, COP1 FPU regs, delay-slot/LL-bit state. |
| `PS2Memory` | `runtime/ps2_memory.h` | PS2 memory map: 32 MB RDRAM, scratchpad, MMIO, GS registers, address translation, DMA cause tracking. |
| `EeScheduler` | `runtime/ee_scheduler.h` | Cooperative EE multitasking: threads, statuses ( `EeThreadStatus` ), waits ( `EeWaitReason` / `EeWaitPayload` — semaphore, event flag, VSync, MPEG), VSync ticks, checkpointing. **This is what runs `run()` .** |

### 4b. Hardware subsystems

| Class | Represents |
|---|---|
| `GS` / `GifArbiter` ( `runtime/gs/` ) | Graphics Synthesizer + GIF packet arbiter; frontends/backends (CPU reference vs `parallel-gs` /Vulkan). |
| `ps2vu::Runtime` / `ps2vu::Unit` ( `runtime/vu/` ) | VU0/VU1 vector units — microprogram memory, interpreter ( `vu_executor` ), macro-mode import/export. |
| `PS2AudioBackend` ( `runtime/ps2_audio.h` ) | SPU2/output; `ps2_audio_vag.cpp` decodes VAG. |
| `PSPadBackend` ( `runtime/ps2_pad.h` ) | Controller input. |
| `PS2RomDevice` ( `runtime/ps2_rom_device.h` ) | CD/DVD device model. |
| `PS2Vfs` ( `runtime/ps2_vfs.h` ) | Virtual file system: `cdrom0:` / `host:` / `mc0:` → host paths. |

### 4c. The "environment" layer (where most porting work happens)

| Layer | Location | Purpose |
|---|---|---|
| Syscalls | `src/lib/Kernel/Syscalls/*` ( `Thread` , `Sync` , `System` , `RPC` , `Interrupt` , `FileIO` , `Deci2` , `Dispatcher` ) | Host implementations of EE kernel calls. `PS2Runtime::handleSyscall` → `ps2_syscalls::dispatchNumericSyscall(id, ...)` . |
| Stubs | `src/lib/Kernel/Stubs/*` ( `SIF` , `GS` , `CD` , `Pad` , `Audio` , `DMA` , `LibC` , `Font` , `MPEG` , `VU` , `TTY` , `MemoryCard` , `IPU` , `Deci2` , `FileIO` , `RPC` , `System` , `Unimplemented` , `Compatibility` ) | Host implementations of SDK libraries. |
| IOP | `src/lib/ps2_iop_host.cpp` , `ps2xIOP/` | `IopSubsystem` (R3000A interpreter + virtual IOP kernel), `PS2IopHostAdapter` , `PS2IopTransport` ; SIF/RPC/DMA bridge between EE and IOP. |

| Layer | Location | Purpose |
|---|---|---|
| Game overrides | `include/game_overrides.h`, `src/lib/game_overrides.cpp` | Per-game, per-build replacements bound by address/CRC during `loadELF` (`PS2_REGISTER_GAME_OVERRIDE`, `bindAddressHandler`). |

**Mental model:** when something breaks, ask "*is the translation wrong, or is the environment incomplete?*" Almost always it is the environment — a stub returning the wrong thing, a missing syscall, a SIF handshake. The project state log is essentially a record of exactly that.

---

## 5. Why there is a generated "functions cpp" file — and how it links

Two things get generated, and both are load-bearing.

### (a) One `.cpp` per translated function

`single_file_output = false` writes ~**53,222** individual files into `data/output/` (e.g. `entry_0x100008.cpp`). Each defines one `void name(...)`.

### (b) `register_functions.cpp` — the dispatch table

`FunctionTableEmitter` produces it. It is the bridge between "a 32-bit guest PC" and "a host function pointer":

```
extern const uint32_t g_ps2RecompiledFunctionTableBase      = 0x100008u;
extern const uint32_t g_ps2RecompiledFunctionTableEnd       = 0x3d1cbcu;
extern const uint32_t g_ps2RecompiledFunctionTableSlotCount = 739117u;
PS2Runtime::RecompiledFunction g_ps2RecompiledFunctionTable[739117u] = {};

namespace {
struct GeneratedFunctionTableInitializer {
    GeneratedFunctionTableInitializer() {
        g_ps2RecompiledFunctionTable[0]   = entry_0x100008;           // 0x100008
        g_ps2RecompiledFunctionTable[73]  = entry_0010012c_0x10012c;  // 0x10012c
        g_ps2RecompiledFunctionTable[138] = FUN_00100230_0x100230;    // 0x100230
        ...
    }
};
static const GeneratedFunctionTableInitializer g_generatedFunctionTableInitializer;
}
```

**How the runtime finds a function** (`ps2_runtime.cpp`):

```
bool generatedFunctionTableSlot(uint32_t address, uint32_t &slot) {
    if ((address & 3u) != 0u || g_ps2RecompiledFunctionTableSlotCount == 0u) return false;
    if (address < Base || address >= End) return false;
    slot = (address - Base) >> 2;   // one slot per 4 bytes
    return slot < SlotCount;
}
```

- `lookupFunction(pc)` → `g_ps2RecompiledFunctionTable[slot]`; if null/out of range, it logs `No exact recompiled function for guest PC ...` and returns a `missingFunction` stub.
- `dispatchGuestBranch(...)` → used by generated `JAL` / `J` sites: looks up the target and **calls it natively** (`targetFn(rdram, ctx, this)`).
- `replaceFunction(addr, fn)` → used by game overrides to swap a broken translation for hand-written C++.

- `include/ps2_recompiled_functions.h` forward-declares every generated function; `ps2_recompiled_stubs.h` declares the stub wrappers.

So the generated files are the **body of the game**, and `register_functions.cpp` is the **address book** that lets the runtime jump into it. Without them the runtime has an empty table and every dispatch reports "missing function."

**Why a table and not direct calls?** The guest computes targets indirectly (function-pointer tables, `jalr`, mid-function resume points), and overrides need to be swappable at runtime. A flat sparse array indexed by `pc >> 2` gives O(1) resolution for any address without a hash lookup.

### How it links — CMake

`ps2xRuntime/CMakeLists.txt`:

```
file(GLOB RUNNER_SRC_FILES CONFIGURE_DEPENDS
    "${CMAKE_CURRENT_SOURCE_DIR}/src/runner/*.cpp")
list(APPEND RUNNER_SRC_FILES ".../src/runner/main.cpp")   # or src/main.cpp
add_executable(ps2EntryRunner ${RUNNER_SRC_FILES})
# unity build batches of 32, PCH, then:
target_link_libraries(ps2EntryRunner PRIVATE ps2_runtime)
```

`ps2_runtime` is the static library (all of `src/lib/...`); `ps2EntryRunner` is **the generated `runner/*.cpp` + `main.cpp`**, linked against it. That is the entire "linking" story.

The TOML in this repo writes to `data/output`, **not** the runtime tree — so there is a copy step: the generated `.cpp` go into `PS2Recomp/ps2xRuntime/src/runner/` and the two headers into `PS2Recomp/ps2xRuntime/include/`. Copy only *content-differing* files so only affected unity batches rebuild.

---

## 6. Making a playable build — the concrete workflow

### One-time prereqs

1. Own the game → obtain `SLUS_212.01` (your own dump). Put it in `data/`.
2. Build the toolchain: `ps2xAnalyzer`, `ps2xRecomp`, `ps2xIOP`, `ps2xRuntime` (CMake 3.20+, C++20, SSE4.1/AVX2). **Use clang-cl + Ninja**; a full rebuild of 53k files on MSVC is 20–30 h.
3. Get function boundaries: run `ps2_analyzer game.elf config.toml`, or (retail game, this repo's path) export `map.csv` from Ghidra and provide `config.toml`.

### Generate

4. Run `ps2_recomp config.toml`. It writes per-function `.cpp`, `register_functions.cpp`, and both headers into `data/output/`.

### Vendor the output into the runtime

5. Copy generated `.cpp` → `PS2Recomp/ps2xRuntime/src/runner/`, and `ps2_recompiled_functions.h` / `ps2_recompiled_stubs.h` → `PS2Recomp/ps2xRuntime/include/`.

### Build

6. `cmake --build PS2Recomp/out/build` (incremental — never a clean build). Kill `ps2EntryRunner.exe` first or the link fails with `LNK1104`.

### Run

7. `run_game_agent.bat 30` → `PS2Recomp/out/build/ps2xRuntime/ps2EntryRunner.exe data/SLUS_212.01`, which runs `main → initialize → loadELF → run → EeScheduler → dispatchGuestBranch` starting at `0x100008`.

### Iterate to "playable" — the four fix tools

1. **TOML** — `stubs` / `skip` / `patch` and *entry points*, then regenerate.
2. **Runtime C++** ( `src/lib/...` ) — implement missing hardware/syscall behavior.
3. **Game override** ( `game_overrides.cpp` ) — replace one broken translated function by address, keyed to the ELF.
4. **Recompiler** — only when the translation itself is wrong (rare).

---

## 7. Where this repo stands

This is a genuine mid-port, not yet "playable".

- ✅ System boundary fixes done/pushed: black-screen/CRT mode, GS interop, SetupThread/async-stack stall, SIFCMD handshake, RPC bind/retry, module version handshake, VU1/VIF1 deadlock.
- ✅ Recompiler discovery gap closed ( `[general].entry_points` added ~358 functions + VU0 macro-mode hooks; 53,222 files reproduced).
- ✅ The game's own IOP modules load and run; sound system (IXAUDIO/DTX) comes up; a real `fileio` HLE service serves the first asset file.
- ⏳ **Current blocker:** the loader stalls on a "subsystem ready" flag at `*(uint64*)0x394378`; its only writer, `FUN_00199938`, is unreachable dead code, and its `WaitSema(65)` peer is never signalled. Displayed content still unconfirmed on screen.

The pipeline is complete and working; what remains is the environment layer — stubs, IOP services, and syscall fidelity — which is where a static-recompilation port spends most of its time.